

Teoria dos Grafos

IMPLEMENTAÇÃO DE GRAFOS:

**ANÁLISE DA DISTRIBUIÇÃO DE COMPONENTES CONEXOS
EM FUNÇÃO DE LIMIARES NA CONSTRUÇÃO DAS ARESTAS**

PARTE 2 (B)

**Alunos: Luciano Kohler e Matheus E. G. Santana
Professor: Gilmário Barbosa dos Santos**

Joinville - 2025
SUMÁRIO

1	INTRODUÇÃO.....	3
2	OBJETIVOS.....	3
3	METODOLOGIA.....	4
3.1	Recursos para o experimento.....	4
3.2	Procedimento experimental.....	4
4	RESULTADOS E DISCUSSÃO.....	6
5	CONCLUSÃO.....	9
6	REFERÊNCIAS BIBLIOGRÁFICAS.....	10

1 INTRODUÇÃO

Um grafo é uma estrutura de dados primordial na área da programação, a mesma é representada por um conjunto de vértices que podem ou não estarem conectados via uma aresta, indicando uma conexão lógica entre os dois elementos. Usualmente, grafos são representados a partir de uma tripla ordenada (N, A, g) , onde 'N' é um conjunto não vazio de nós (vértices), 'A' é um conjunto de arcos (arestas), que correspondem a pares não ordenados, e 'g' é uma função que associa a cada arco um par não ordenado x-y de nós, chamados de extremidades de 'a'.

Neste relatório, também será usado o conceito de “matriz de adjacência”, que se refere a um método de representar um grafo computacionalmente. Essa representação ocorre por meio de uma matriz $n \times n$, com n correspondendo ao número de vértices pertencentes ao grafo, e onde cada elemento $a[i][j]$ corresponde à quantidade de arestas que um vértice i possui para o vértice j. Segue uma imagem para auxílio da explicação:

	v1	v2	v3	v4
v1	1	0	1	1
v2	0	0	0	1
v3	1	0	0	1
v4	1	1	1	0

Matriz de adjacência do grafo G

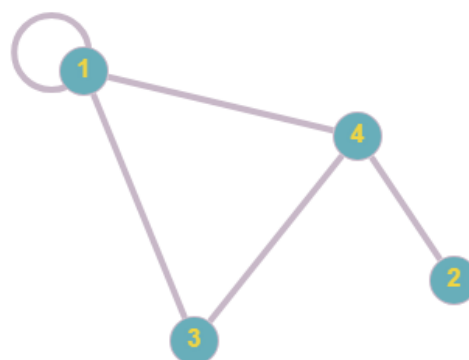


Figura 1^[1]: Imagem ilustrando o grafo G

Em grafos, existe o conceito de “grupos conexos”, ou “clusters”, que são separações dos vértices em grupos individuais compostos por outros vértices que ocorrem geralmente quando se é trabalhado com um grande volume de nodos (nossos dados), que possuem algum tipo de correlação entre si. Uma análise desses grupos conexos, principalmente em bases de dados mais robustas, pode vir a se provar muito benéfica para encontrar otimizações e padrões de comportamento entre os vértices analisados.

2 OBJETIVOS

Após a realização da parte A do trabalho, envolvendo a criação do algoritmo que *clusteriza* uma base de dados quadridimensional, uma nova parte do estudo será desenvolvida, com o intuito de, à partir do programa anterior, implementar funcionalidades novas para análise dos clusters:

- **Desenvolver** um algoritmo que automaticamente encontra um limiar que consiga separar a nova base de dados em três clusters por meio de uma busca binária
- **Encontrar** o centro de gravidade de cada cluster

- **Mostrar** a taxa de acerto e erros (ht/miss rate) de cada cluster, sendo que cada nodo possui um tipo intrínseco que pode ou não condizer com a classificação de cada cluster

3 METODOLOGIA

3.1 Recursos para o experimento

- Utilização de um notebook Dell Inspiron 15:
 - Memória RAM: 16GB
 - Processador: Intel Core i5-1235U 10 núcleos
 - Tipo de unidade de armazenamento: SSD M.1 512GB
- Compilação do código em C feita à partir do GCC versão 14.2.0 e executado via linha de comando no terminal (.\\main.exe)
- Bibliotecas utilizadas:
 - stdio.h e stdlib.h (padrões de qualquer código C)
 - string.h para manipulação de strings e leitura de dados
 - math.h para o cálculo das distâncias euclidianas
- Implementação da estrutura de grafo contendo:
 - Coordenadas (quádrupla)
 - O tipo que cada nodo tem em um cenário ideal
 - Distâncias de nodo a nodo
 - Distâncias normalizadas (variando de 0 a 1)
 - Matriz de adjacência para cada limiar analisado (que muda de iteração em iteração)
- Utilização de leitura e escrita de dados em arquivos CSV, contendo na pasta input, dois arquivos **.csv** contendo coordenadas de 150 vértices, e, na pasta output, o arquivo **adjacencias.csv**, armazenando a persistência da matriz de adjacência da amostra, e o arquivo **clusters.txt**, armazenando as informações sobre os clusters encontrados.

3.2 Procedimento experimental

O processo de análise foi inteiramente feito em um único arquivo C chamado **main.c**, onde nele, foram seguidos os seguintes passos:

1. **Declaração de variáveis e escolha do arquivo de entrada**, onde todas as variáveis necessárias para o experimento foram declaradas em relação ao tamanho da amostra (150) e a escolha entre as duas amostras é feita pelo usuário.
2. **Leitura do arquivo de entrada**, onde por meio do dataset escolhido, foram recebidas e salvas as coordenadas e o tipo de cada nodo.
3. **Cálculo de distâncias e procura da distância máxima e mínima**, onde para cada par de nodos, foi calculada a distância euclidiana deles e, se a distância entre

ambos for menor que a mínima ou maior que a máxima, este valor também é salvo para uso posterior no passo de cálculo de distâncias normalizadas. A fórmula^[2] utilizada para o cálculo de distâncias euclidianas é a seguinte:

$$d = \sqrt{\sum_{i=1}^n (b_i - a_i)^2}$$

Figura 2: Fórmula para o cálculo da distância euclidiana de dois pontos em uma dimensão n qualquer.

4. **Cálculo de distâncias normalizadas**, onde por meio da fórmula de normalização de um conjunto de dados^[3], foram normalizadas as distâncias, fazendo com que as mesmas variem de 0 a 1. A fórmula utilizada para o cálculo das distâncias normalizadas é a seguinte:

$$\hat{X} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Figura 3: Fórmula para o cálculo da normalização de um conjunto de dados qualquer.

5. **Iteração para a procura do limiar ideal**, onde por meio de uma busca binária (com limitantes sendo 0 e 1), é encontrado um limiar que separa o dataset em três clusters distintos.
 - 5.1. **Cálculo do centro de gravidade dos clusters**, onde, após a determinação do cluster, fazemos uma média das coordenadas (x,y,z,w) dos nodos pertencentes ao mesmo, para, assim, determinar o centro de gravidade da nuvem de vértices.
6. **Análise dos dados com o limiar dado e salvamento dos dados**, por fim, com os dados de cada cluster salvos, é printado as características do cluster, incluindo informações como quantidade de acertos e erros (Verdadeiro Positivo [VP], Verdadeiro Negativo [VN], Falso Positivo [FP] e Falso Negativo [FN]), acurácia adquirida (calculada pela fórmula abaixo), e outras informações, que são salvas no arquivo **clusters.txt** para acesso futuro.

$$acurácia = \frac{VP + VN}{VP + FN + VN + FP}$$

Figura 4: Fórmula^[5] para o cálculo da acurácia, utilizando a quantidade de VP, VN, FN e FP.

4 RESULTADOS E DISCUSSÃO

Após a devida realização dos experimentos, foi possível notar que, para o dataset número 1 (que possui nodos relativamente separados entre si, deixando a formação de clusters mais evidente), adquirimos os seguintes resultados de acordo com a implementação feita:

```
ANALISE DOS CLUSTERS:
Cluster num. 1:
  Quantidade de elementos no cluster: 51
  Centro de gravidade do cluster: (5.93, 2.78, 4.28, 1.35)
  Tipo do cluster: Tipo 1
  Quantidade de acertos: 50
  Quantidade de erros: 1
  Taxa de acerto: 98.04%
-----
Cluster num. 2:
  Quantidade de elementos no cluster: 50
  Centro de gravidade do cluster: (5.01, 3.42, 1.46, 0.24)
  Tipo do cluster: Tipo 2
  Quantidade de acertos: 50
  Quantidade de erros: 0
  Taxa de acerto: 100.00%
-----
Cluster num. 3:
  Quantidade de elementos no cluster: 49
  Centro de gravidade do cluster: (7.42, 3.78, 6.37, 2.83)
  Tipo do cluster: Tipo 3
  Quantidade de acertos: 49
  Quantidade de erros: 0
  Taxa de acerto: 100.00%
-----
MATRIZ DE CONFUSAO:
      1    2    3
1    50    0    1
2     0   50    0
3     0    0   49
-----
ACURACIA TOTAL: 99.33%
-----
```

Figura 5: Resultados impressos no terminal durante a execução do programa no dataset 1.

Visualmente, o histograma abaixo representa os clusters e a quantidade de elementos em cada um:

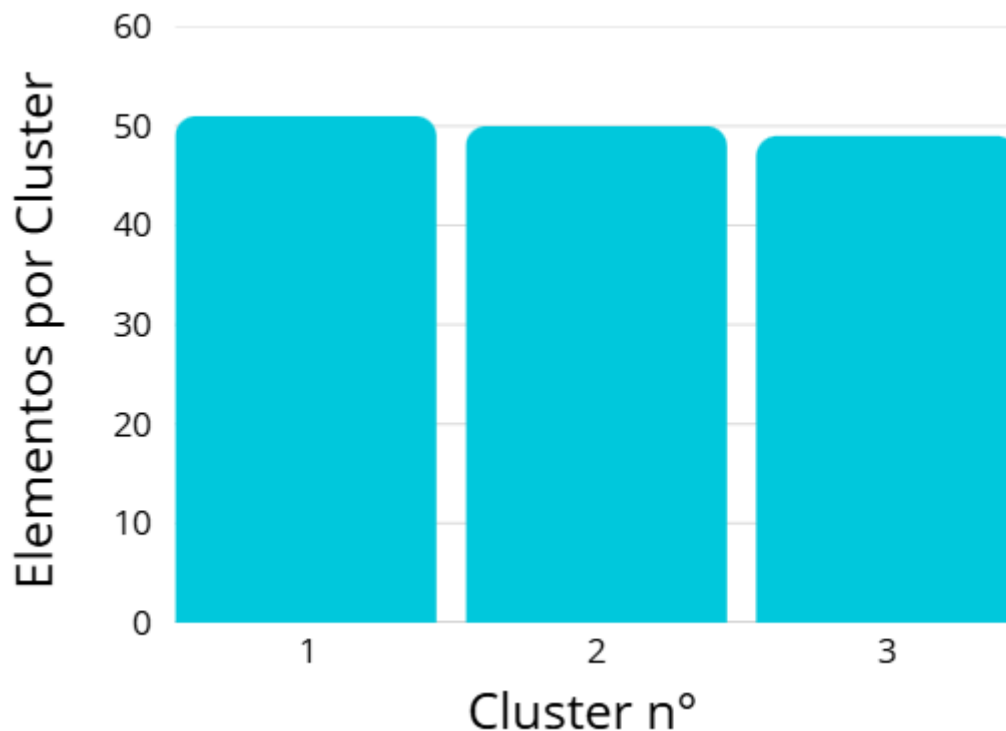


Figura 6: Histograma representante da quantidade de nodos por cluster no dataset 1

Analisando estes resultados, podemos perceber que, com o método de busca binária, conseguimos adquirir um resultado muito satisfatório para a divisão dos clusters, com apenas um FP presente no Cluster num. 1, e adquirindo uma acurácia total de 99.33%.

Podemos também observar pelo centro de gravidade do cluster (informação requisitada pelo professor nas instruções do trabalho) que as nuvens estão relativamente distantes entre si (característica do primeiro dataset, tornando mais simples a separação em três clusters com a mesma quantidade de elementos).

Além disso, temos também o limiar adquirido é aproximadamente 0.107422, mostrado nos valores adquiridos pelas iterações da busca binária.

Junto da escolha do dataset 1, podemos escolher o dataset 2 para aplicar o algoritmo implementado, obtendo os seguintes resultados:

```
ANALISE DOS CLUSTERS:
Cluster num. 1:
  Quantidade de elementos no cluster: 98
  Centro de gravidade do cluster: (6.58, 3.20, 5.22, 2.02)
  Tipo do cluster: Tipo 1
  Quantidade de acertos: 50
  Quantidade de erros: 48
  Taxa de acerto: 51.02%
-----
Cluster num. 2:
  Quantidade de elementos no cluster: 50
  Centro de gravidade do cluster: (5.01, 3.42, 1.46, 0.24)
  Tipo do cluster: Tipo 2
  Quantidade de acertos: 50
  Quantidade de erros: 0
  Taxa de acerto: 100.00%
-----
Cluster num. 3:
  Quantidade de elementos no cluster: 2
  Centro de gravidade do cluster: (8.60, 4.60, 7.35, 2.90)
  Tipo do cluster: Tipo 3
  Quantidade de acertos: 2
  Quantidade de erros: 0
  Taxa de acerto: 100.00%
-----
MATRIZ DE CONFUSAO:
      1      2      3
1     50      0     48
2      0     50      0
3      0      0      2
-----
ACURACIA TOTAL: 68.00%
-----
```

Figura 7: Resultados impressos no terminal durante a execução do programa no dataset 2.

Tal qual o dataset 1, também existe um histograma que representa os clusters e a quantidade de elementos em cada um:

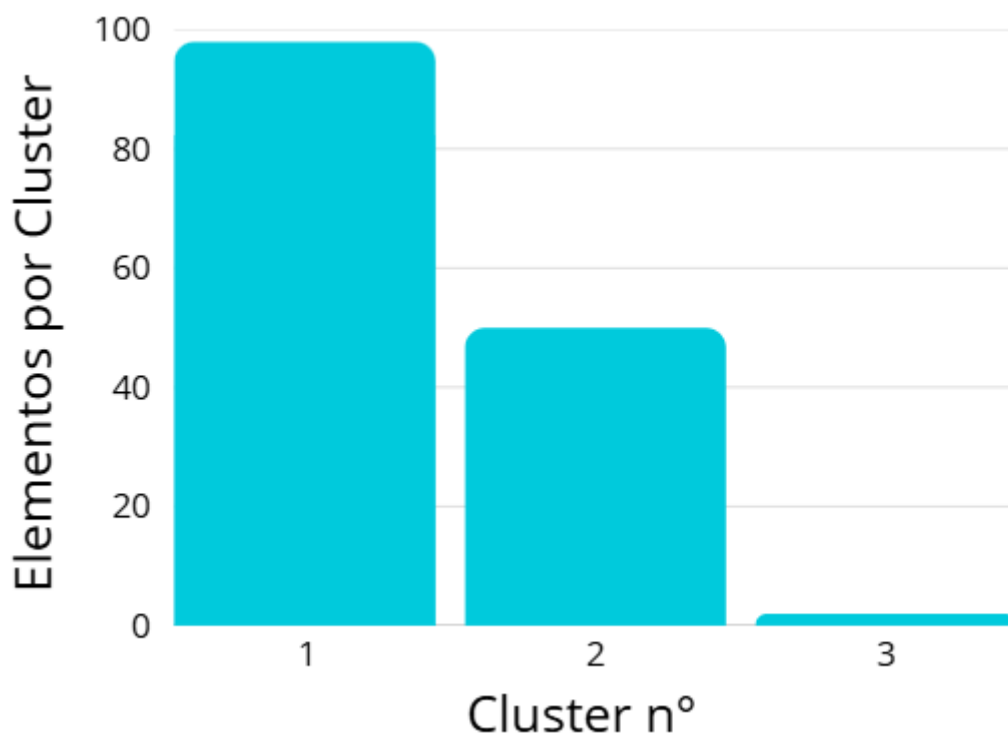


Figura 8: Histograma representante da quantidade de nodos por cluster no dataset 2.

Analisando os resultados deste dataset, podemos observar que os resultados não foram muito satisfatórios, com uma acurácia total de 68%, onde basicamente duas das nuvens estão juntas. Isso se dá pelo fato de que o dataset possui vértices estrategicamente colocados entre as duas nuvens (pontes) que acabam por conectar os clusters.

Contudo, dado as condições adquiridas pela impressão dos dados no dataset, podemos perceber que, caso elevarmos o nosso limiar, vamos apenas juntar os grupos, ou seja, Cluster num. 3 se juntaria com o Cluster num. 1 (já que o Cluster num 1 é um cluster que possui apenas vértices do tipo 1 e 3), e, no pior dos casos, obteríamos um único cluster com 150 nodos.

Por outro lado, caso reduzirmos o limiar, iremos criar mais grupos, já que o grupo com 2 vértices ainda existirá (no pior dos casos, também irá se separar), e queremos apenas separar a nuvem do Tipo 1 da nuvem do Tipo 3, sem criar mais um novo grupo (o que é impossível nessa situação). Logo, este é o melhor resultado que conseguimos obter no dataset disponibilizado.

5 CONCLUSÃO

À partir da análise, é possível inferir que é necessário apenas uma pequena quantidade de “nodos maliciosos” para que a clusterização da amostra tome um rumo totalmente diferente do planejado, a melhor solução para este problema seria a modificação ou deleção dos nodos que complicam a dinâmica do conjunto, porém essa prática não é recomendada, já que, a manipulação de dados nunca é vantajosa na área da análise de dados, afinal, um dado manipulado não dá valor algum para a inferência de atitudes.

6 REFERÊNCIAS BIBLIOGRÁFICAS

- [1] Graph Online. *Criando um grafo a partir de uma matriz de adjacência*. **Graph Online**, [s.d.]. Disponível em: https://graphonline.top/pt/create_graph_by_matrix. Acesso em: 13 nov. 2025.
- [2] Vinod Chugani. *Entendendo a distância euclidiana: Da teoria à prática*. **DataCamp**, 1 de out. de 2024. Disponível em: <https://www.datacamp.com/pt/tutorial/euclidean-distance>. Acesso em: 13 nov. 2025.
- [3] Aashish Nair. *Standardization vs normalization*. **Towards Data Science**, 21 mar. 2022. Disponível em: <https://towardsdatascience.com/standardization-vs-normalization-dc81f23085e3/>. Acesso em: 13 nov. 2025.
- [4] GEEKSFORGEES. *Depth First Search or DFS for a Graph*. **GeeksforGeeks**, 3 out. 2025. Disponível em: <https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/>. Acesso em: 13 nov. 2025.
- [5] Diego Mariano. *Métricas de avaliação em machine learning*. **Revista Bioinfo**, 19 jun. 2021. Disponível em: <https://bioinfo.com.br/metricas-de-avaliacao-em-machine-learning-acuracia-sensibilidade-precisao-especificidade-e-f-score/>. Acesso em: 14 nov. 2025.